

Documentation technique — Agent IA d'entraînement aux ouvertures (POC FFE)

Rôle de ce document : tout le détail technique que la présentation de soutenance ne porte pas (volontairement — la présentation raconte la démarche, la documentation porte les détails). Chaque section renvoie vers la source faisant foi dans le dépôt : code, notebooks exécutés, documents de conception. **Dépôt** : <https://github.com/richardhugou/p13-agent-ia-echecs> · **Démarrage** : `./demarrer.sh` (README)

1. Vue d'ensemble

Sept conteneurs orchestrés par docker-compose, un modèle local (Ollama) sur la machine hôte :

Service	Techno	Rôle	Port hôte
frontend	Angular 19 + ngx-chess-board, servi par nginx (multi-étages, image 78 Mo)	échiquier + panneau coach	4200
api	FastAPI + LangGraph + Stockfish embarqué (arm64)	l'agent	8000
milvus (+ etcd, minio)	Milvus 2.4 standalone	recherche vectorielle (index HNSW, cosinus)	19530 (job ETL)
mongodb	Mongo 7	caches (explorer 24 h, évals sans TTL, vidéos 7 j)	interne
mlflow	MLflow 2.22	tracking des runs d'évaluation	5001

LLM et embeddings tournent sur l'hôte via **Ollama** (qwen3.5:4b, qwen3-embedding:0.6b) — décisions D1/D-a prises sur mesures (voir §7). Schémas complets : `docs/05-architecture-technique.md`.

2. L'agent — graphe LangGraph

```
valider_fen → identifier_ouverture → [routeur déterministe : ≥ 5 parties masters ?]
    oui → coups_theoriques (Lichess explorer)          non → evaluer_position (Stockfish)
        → contexte_rag (Milvus, règle des rayons) → videos (YouTube) → synthese (LLM)
```

- **Règle d'or** : le LLM n'est jamais la source de vérité — coups = stats Lichess filtrées par python-chess (0 illégal par construction), évaluations = Stockfish, **sources ajoutées par le code** (pas par le LLM).
- **Routeur déterministe** : seuil `THEORY_MIN_GAMES=5` — testé aux bornes (4/5/6), pas un choix de LLM.
- **Fallback par nœud** : Lichess KO → moteur+RAG ; Milvus KO → réponse sans fiches + note d'incident (observé en conditions réelles) ; YouTube KO → sans vidéos ; LLM KO → gabarit déterministe toujours juste.
- Code : `backend/graph/` (nodes, state, synthese, notation) — 65 tests, `backend/tests/`.

3. La base vectorielle (le cœur documentaire)

Pourquoi : retrouver les passages encyclopédiques qui « parlent de la même chose » qu'une question d'élève, y compris entre français et anglais — chose impossible avec une recherche par mots-clés (« pourquoi le fou vise f7 » ne contient pas « Giuoco Piano »).

Comment on indexe : 1. Corpus signé (`etl/corpus.yml` : 161 pages Wikipédia FR + Wikibooks EN — règle : pas de manifeste signé, pas d'extraction) ; 2. Transformation : nettoyage wikitext, chunking par section 300–500 tokens + recouvrement 15 % + fil d'Ariane, FEN de référence calculé, déduplication → **477 fiches** ; 3. Vectorisation : chaque fiche → un vecteur de **1024 nombres** (Qwen3-Embedding-0.6B via Ollama) qui encode son sens ; FR et EN partagent le même espace (l'argument décisif du corpus mixte) ; 4. Insertion Milvus : collection `openings_kb`, index **HNSW**, métrique **cosinus** (l'angle entre deux vecteurs : 1 = même sens, 0 = sans rapport), métadonnées scalaires (`eco`, `ouverture`, `source_url`, `lang...`).

Comment on cherche : question → vecteur (avec **préfixe d'instruction sur les requêtes uniquement** — mesuré : séparation cible/hors-sujet 0,29 → 0,50, notebook 02) → les k=5 fiches à plus fort cosinus, **filtrées par rayon** :

- **Règle des rayons signés** (décision du 26/08, notebook 07) : le corpus n'est consulté que dans un rayon établi par la position (code ECO → rayon) ou par le nom d'ouverture dans la question (alias FR/EN). Ouverture hors des 8 rayons → zéro fiche + réponse honnête. Une citation trompeuse est **impossible par construction**.
- **Seuil filet** `RAG_SCORE_MIN=0.58` : coupe les hors-sujet grossiers à l'intérieur d'un rayon.

Modèle de données complet (champs Milvus + collections MongoDB) : `docs/05-architecture-technique.md` §modèle ; ETL : `etl/README` et scripts `extraire/transformer/charger.py`.

4. L'API

Endpoint	Rôle
GET <code>/api/v1/healthcheck</code>	santé du service + Mongo

Endpoint	Rôle
GET /api/v1/moves?fen=	coups théoriques + stats + san_fr (« Fc5 (fou f8) »)
GET /api/v1/evaluate?fen=	évaluation Stockfish (cp/mate, meilleure ligne), cache par FEN
GET /api/v1/vector-search?q=&k=&eco=	recherche vectorielle diagnostic (scores bruts, seuil désactivé)
GET /api/v1/videos?...	vidéos YouTube filtrées (durée 4–30 min, titre), cache 7 j
POST /api/v1/agent/ask {fen, question?}	le parcours complet du graphe — réponse structurée en blocs + sources

Erreurs typées et actionnables (429 → Retry-After, 401 → message explicite, corps d'erreur porté). FEN en query param (choix documenté, docs/05 §3). Swagger : <http://localhost:8000/docs>.

5. Le front

Angular 19 (starter OC material-chessboard, lib ngx-chess-board locale). Parcours élève (revu sur retours mentor) : choix du camp (le plateau se retourne) → **mode entraînement : l'agent joue les coups de l'adversaire** (le plus joué des maîtres via /moves — jamais un choix de LLM ; plus de théorie → signal et main rendue à l'élève) → « Annuler le coup » retire la paire de coups → sélecteur d'ouverture (8 rayons) → **bouton « Lancer l'IA »** (pas de déclenchement automatique). **Un seul jeu de pièces** : les pièces adverses sont verrouillées (elles ne se déverrouillent qu'à la sortie de théorie, pour saisir le coup réel de l'adversaire). **Les suggestions s'affichent sur le plateau** (flèches vertes, top 3 théorique) ; séquence : conseils + flèches d'abord, réponse adverse ensuite. Notation française annotée « Fc5 (fou f8) ». Lien profond de démo : <http://localhost:4200/#ouverture=Italienne>.

6. Évaluation et mesures (tout est rejouable)

Règle de labo : **aucune mesure ne vit uniquement dans une discussion** — chaque chiffre sort d'un notebook exécuté versionné ou d'un run MLflow (expérience gold-set-rag, <http://localhost:5001>).

Mesure	Valeur	Source rejouable
Coups illégaux affichés	0 / 56	notebooks/05-mesures-agent.ipynb (scénario de démo rejoué)
recall@5 · MRR (gold set 25 questions figé)	1,0 · 1,0	evaluation/evaluer.py → runs MLflow A_naif / B_soigne
Abstention sur les 5 pièges	5/5 par construction	notebooks 05 & 07 + test_rag_service.py / test_graph_nodes.py
Réponses sourcées	100 % par construction	code (graph/synthese.py)

Mesure	Valeur	Source rejouable
Latence recherche vectorielle p95	7–11 ms à chaud	notebook 04 / MLflow
Latence agent p95 (LLM local compris)	6,1 s (p50 4,5 s)	notebook 05, figure 06
Séparation embeddings (préfixe d'instruction)	0,29 → 0,50	notebook 02 + <code>test_embeddings_mesure.py</code>
Installation fraîche	app 2 min 09 , bibliothèque 2 min 28	<code>tester-installation.sh</code>
Coût LLM total	0,00 € (local)	notebook 05

Leçon documentée : le gold set v1 (labels au niveau rayon) est trop grossier pour départager les chunkings — recall 1,0 partout ; le gold set v2 à labels fins est l'axe assumé. Histoire complète de la décision d'abstention (seuil 0,63 → 0,67 refusé sur mesure → règle des rayons) : notebook 07 + `suivi/` (privé).

7. Décisions techniques (chacune avec sa mesure)

Décision	Choix	Preuve
D1 — LLM de synthèse	qwen3.5:4b local (3,2 Go RAM, 3–7 s, 0 €) ; API Anthropic = option (LLM_PROVIDER) ; gabarit = repli	campagne 4 modèles (journal 22/08)
D-a — Embeddings	qwen3-embedding:0.6b via Ollama, préfixe requêtes seulement	notebook 02
Routeur théorie/moteur	déterministe, seuil 5 parties	tests aux bornes
Abstention RAG	règle des rayons signés + filet 0,58	notebooks 05/07
Corpus	manifeste signé 8 ouvertures, 161 pages	<code>etl/corpus.yml</code>
Garde-fous LLM	faits annotés (« Fc5 — le Fou va en c5 »), temp 0,2, think off	<code>graph/notation.py</code> , tests

8. Exploitation

- **Démarrage** : `./demarrer.sh` (Ollama vérifié + modèles + `.env` + compose + attente santé et bibliothèque). Jamais `docker restart` après un changement de `.env` (non relu) — recréer avec `compose up -d`.
- **Chargement du corpus** : `cd etl && uv run extraire.py && uv run transformer.py && uv run charger.py` (~10 min, idempotent).
- **Test d'installation** : `./tester-installation.sh`. **Secrets** : `.env` git-ignoré (LICHESS_API_TOKEN requis, YOUTUBE_API_KEY optionnelle).
- **CI** : GitHub Actions — lint (ruff) + 65 tests backend, build frontend. Gitflow `main` ← `develop` ← `feature/<Nom>`.

- Pièges connus documentés : Milvus compte les varchar en octets ; port hôte 5000 squatté par AirPlay (macOS) → MLflow sur 5001 ; Milvus recharge ses collections après redémarrage (attente intégrée au script).